

Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 13 tháng 7 2024, 10:56 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:34 PM
Thời gian thực hiện	41 Các ngày 4 giờ
Điểm	7,00/7,00
Điểm	10,00 trên 10,00 (100%)



Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

[Eng] Given a queue of integers of even length, rearrange the elements by interleaving the first half of the queue with the second half of the queue.

Your task is to implement `interleaveQueue` function.

`stack` and `queue` are included.

[Vie] Cho 1 hàng đợi có số lượng phần tử là số chẵn, sắp xếp lại các phần tử theo quy tắc xen kẽ phần tử ở nửa đầu và nửa sau của hàng đợi.

Sinh viên cần hiện thực hàm `interleaveQueue`.

Thư viện `stack` và `queue` đã được thêm vào.

For example:

Test	Input	Result
<pre>queue<int> q; int n; cin >> n; for (int i = 0; i < n; i++){ int element; cin >> element; q.push(element); } interleaveQueue(q); while (!q.empty()){ cout << q.front() << ' '; q.pop(); }</pre>	<pre>4 1 2 3 4</pre>	<pre>1 3 2 4</pre>
<pre>queue<int> q; int n; cin >> n; for (int i = 0; i < n; i++){ int element; cin >> element; q.push(element); } interleaveQueue(q); while (!q.empty()){ cout << q.front() << ' '; q.pop(); }</pre>	<pre>6 2 4 6 8 10 12</pre>	<pre>2 8 4 10 6 12</pre>

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <queue>
2 #include <stack>
3 #include <iostream>
4
5 void interleaveQueue(std::queue<int>& q) {
6     if (q.empty()) {
7         return;
8     }
9
10    std::stack<int> s;
11    int n = q.size();
12    int halfSize = n / 2;
13
14    // Di chuyển nửa đầu của hàng đợi vào stack
15    for (int i = 0; i < halfSize; ++i) {
16        s.push(q.front());
17        q.pop();
18    }
```

```

19
20 // Di chuyển lại các phần tử từ stack vào hàng đợi
21 while (!s.empty()) {
22     q.push(s.top());
23     s.pop();
24 }
25
26 // Di chuyển nửa đầu của hàng đợi (đã đảo ngược) ra sau cùng của hàng đợi
27 for (int i = 0; i < halfSize; ++i) {
28     q.push(q.front());
29     q.pop();
30 }
31
32 // Di chuyển nửa sau của hàng đợi vào stack
33 for (int i = 0; i < halfSize; ++i) {
34     s.push(q.front());
35     q.pop();
36 }
37
38 // Xen kẽ các phần tử từ stack và hàng đợi
39 while (!s.empty()) {
40     q.push(s.top());
41     s.pop();
42     q.push(q.front());
43     q.pop();
44 }
45 }

```

	Test	Input	Expected	Got	
✓	<pre> queue<int> q; int n; cin >> n; for (int i = 0; i < n; i++){ int element; cin >> element; q.push(element); } interleaveQueue(q); while (!q.empty()){ cout << q.front() << ' '; q.pop(); } </pre>	<pre> 4 1 2 3 4 </pre>	1 3 2 4	1 3 2 4	✓
✓	<pre> queue<int> q; int n; cin >> n; for (int i = 0; i < n; i++){ int element; cin >> element; q.push(element); } interleaveQueue(q); while (!q.empty()){ cout << q.front() << ' '; q.pop(); } </pre>	<pre> 6 2 4 6 8 10 12 </pre>	2 8 4 10 6 12	2 8 4 10 6 12	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 2

Đúng

Đạt điểm 1,00 trên 1,00

Research **queue** which is implemented in C library at <http://www.cplusplus.com/reference/queue/queue/>. You can use library **queue** in c++ for this question.

Using **queue**, complete function **bool isBipartite(vector<vector<int>> graph)** to determine if a [graph](#) is bipartite or not (the [graph](#) can be disconnected). In caat https://en.wikipedia.org/wiki/Bipartite_graph.

You can use below libraries in this question.

```
#include <iostream>
#include <vector>
#include <queue>
```

For example:

Test	Result
<pre>int G[6][6] = { {0, 1, 0, 0, 0, 1}, {1, 0, 1, 0, 0, 0}, {0, 1, 0, 1, 0, 0}, {0, 0, 1, 0, 1, 0}, {0, 0, 0, 1, 0, 1}, {1, 0, 0, 0, 1, 0} }; int n = 6; vector<vector<int>> graph(n, vector<int>()); for (int i = 0; i < n; ++i) { for (int j = 0; j < n; ++j) { if (G[i][j]) graph[i].push_back(j); } } isBipartite(graph) ? cout << "Yes" : cout << "No";</pre>	Yes

Answer: (penalty regime: 0 %)

Reset answer

```
1 bool isBipartite(vector<vector<int>> graph) {
2     int n = graph.size();
3     vector<int> color(n, -1); // Initialize all vertices with no color
4
5     for (int i = 0; i < n; ++i) {
6         if (color[i] == -1) { // Not yet colored
7             queue<int> q;
8             q.push(i);
9             color[i] = 0; // Color the first node with color 0
10
11            while (!q.empty()) {
12                int node = q.front();
13                q.pop();
14
15                for (int neighbor : graph[node]) {
16                    if (color[neighbor] == -1) {
17                        // Color with opposite color
18                        color[neighbor] = 1 - color[node];
19                        q.push(neighbor);
20                    }
21                }
22            }
23        }
24    }
25    return true;
26 }
```

```
20 } else if (color[neighbor] == color[node]) {  
21     // If the neighbor has the same color, it's not bipartite  
22     return false;  
23 }  
24 }  
25 }  
26 }  
27 }  
28 }  
29 return true;  
30 }
```

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 3

Đúng

Đạt điểm 1,00 trên 1,00

Research **queue** which is implemented in C library at: <http://www.cplusplus.com/reference/queue/queue/>. You can use library **queue** in c++ for this question.

Using **queue**, complete function **void bfs(vector<vector<int>> graph, int start)** to traverse all the nodes of the **graph** from given start node using Breadth First **Search** algorithm and data structure **queue**, and print the order of visited nodes.

You can use below libraries in this question.

```
#include <iostream>
#include <vector>
#include <queue>
```

For example:

Test	Result
<pre>int init_graph[10][10] = { {0, 1, 1, 0, 1, 0, 1, 0, 1, 0}, {0, 0, 1, 1, 0, 0, 0, 1, 0, 0}, {0, 1, 0, 0, 0, 1, 1, 0, 1, 1}, {1, 0, 0, 0, 0, 0, 0, 1, 0, 0}, {0, 1, 0, 0, 0, 0, 0, 1, 0, 0}, {1, 0, 1, 0, 1, 0, 0, 0, 1, 0}, {0, 0, 1, 1, 0, 1, 0, 0, 0, 0}, {1, 0, 0, 0, 0, 1, 1, 0, 1, 0}, {0, 0, 0, 0, 0, 1, 0, 1, 0, 1}, {1, 0, 1, 0, 1, 0, 0, 0, 1, 0} }; int n = 10; vector<vector<int>> graph(n, vector<int>()); for (int i = 0; i < n; ++i) { for (int j = 0; j < n; ++j) { if (init_graph[i][j]) graph[i].push_back(j); } } bfs(graph, 0);</pre>	0 1 2 4 6 8 3 7 5 9

Answer: (penalty regime: 0 %)

Reset answer

```
1 void bfs(vector<vector<int>> graph, int start) {
2     int n = graph.size();
3     vector<bool> visited(n, false); // Đánh dấu các đỉnh đã được thăm
4     queue<int> q; // Hàng đợi để duyệt BFS
5
6     // Bắt đầu từ đỉnh start
7     visited[start] = true;
8     q.push(start);
9
10    while (!q.empty()) {
11        int node = q.front();
12        q.pop();
13        cout << node << " ";
14
15        // Duyệt tất cả các đỉnh kề
16        for (int neighbor : graph[node]) {
17            if (!visited[neighbor]) {
18                visited[neighbor] = true;
19                q.push(neighbor);
20            }
21        }
22    }
```

23 }

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 4

Đúng

Đạt điểm 1,00 trên 1,00

Implement all methods in class **Queue** with template type **T**. The description of each method is written as comment in frame code.

```
#ifndef QUEUE_H
#define QUEUE_H
#include "DLinkedList.h"
template<class T>
class Queue {
protected:
    DLinkedList<T> list;
public:
    Queue() {}
    void push(T item) ;
    T pop() ;
    T top() ;
    bool empty() ;
    int size() ;
    void clear() ;
};

#endif /* QUEUE_H */
```

You can use all methods in class **DLinkedList** without implementing them again. The description of class **DLinkedList** is written as comment in frame code.

```
template <class T>
class DLinkedList
{
public:
    class Node;    //forward declaration
protected:
    Node* head;
    Node* tail;
    int count;
public:
    DLinkedList() ;
    ~DLinkedList();
    void add(const T& e);
    void add(int index, const T& e);
    T removeAt(int index);
    bool removeItem(const T& removeItem);
    bool empty();
    int size();
    void clear();
    T get(int index);
    void set(int index, const T& e);
    int indexOf(const T& item);
    bool contains(const T& item);
};
```

For example:



Test	Result
Queue<int> queue; assert(queue.empty()); assert(queue.size() == 0);	

Answer: (penalty regime: 0 %)

Reset answer

```
1 void push(T item) {  
2     // TODO: Push new element into the end of the queue  
3     this->list.add(item);  
4 }  
5  
6 T pop() {  
7     // TODO: Remove an element in the head of the queue  
8     return this->list.removeAt(0);  
9     .....  
10 }  
11  
12 T top() {  
13     // TODO: Get value of the element in the head of the queue  
14     return this->list.get(0);  
15     .....  
16 }  
17  
18 bool empty() {  
19     // TODO: Determine if the queue is empty  
20     return this->list.empty();  
21     .....  
22 }  
23  
24 int size() {  
25     // TODO: Get the size of the queue  
26     return this->list.size();  
27     .....  
28 }  
29  
30 void clear() {  
31     // TODO: Clear all elements of the queue  
32     list.clear();  
33 }
```



Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 5

Đúng

Đạt điểm 1,00 trên 1,00

A nice number is a positive integer that contains only 2's and 5's.

Some nice numbers are: 2, 5, 22, 25, 52, 55, ...

Number 2 is the first nice number.

Given an integer N, return the Nth nice number.

Note: iostream, vector, queue are already included for you.

Constraint:

$1 \leq n \leq 10^6$

Example 1:

Input:

n = 5

Output:

52

Explanation:

The sequence of nice numbers is 2, 5, 22, 25, 52, 55, ...

The 5th number in this sequence is 52

Example 2:

Input:

n = 10000

Output:

2255522252225

For example:

Test	Input	Result
<pre>int n; cin >> n; cout << nthNiceNumber(n) << endl;</pre>	5	52
<pre>int n; cin >> n; cout << nthNiceNumber(n) << endl;</pre>	10000	2255522252225

Answer: (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
1 long long nthNiceNumber(int n) {
2     if (n <= 0) return 0;
3
4     queue<string> q;
5     q.push("2");
6     q.push("5");
7
8     string current;
9
10    for (int i = 0; i < n; ++i) {
11        current = q.front();
12        q.pop();
13
14        q.push(current + "2");
15        q.push(current + "5");
```

```
15     q.push(current + 5),
16 }
17
18 return stoll(current);
19 }
```

	Test	Input	Expected	Got	
✓	int n; cin >> n; cout << nthNiceNumber(n) << endl;	5	52	52	✓
✓	int n; cin >> n; cout << nthNiceNumber(n) << endl;	10000	2255522252225	2255522252225	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 6

Đúng

Đạt điểm 1,00 trên 1,00

Given a $n*m$ grid where each cell in the grid can have a value of 0, 1 or 2, which has the following meaning:

- 1. Empty cell
- 2. This cell contains a fresh apple
- 3. This cell contains a rotten apple

After 1 second, the cell with rotten apple will rot all fresh apples in all the cells adjacent to it (i.e the cells $(x+1, y)$, $(x-1, y)$, $(x, y+1)$, $(x, y-1)$)

Determine the minimum time (in seconds) required to rot all apples. If this cannot be done, return -1.

Note: iostream, vector, and queue are already included.

Constraint:

$1 \leq n, m \leq 500$

Hint: Have you ever heard about [breadth-first-search](#)?

Example 1:

Input: grid = {{2,2,0,1}}

Output: -1

Explanation:

The grid is

2 2 0 1

The apple at (0, 3) cannot be rotten

Example 2:

Input: grid = {{0,1,2},{0,1,2},{2,1,1}}

Output: 1

Explanation:

The grid is

0 1 2

0 1 2

2 1 1

Apples at positions (0,2), (1,2), (2,0)

will rot apples at (0,1), (1,1), (2,2) and (2,1) after 1 second.

For example:

Test	Input	Result
<pre>int rows, cols; cin >> rows >> cols; vector<vector<int>> grid(rows, vector<int>(cols)); for(int i = 0; i < rows; i++) { for(int j = 0; j < cols; j++) cin >> grid[i][j]; } cout << secondsToBeRotten(grid);</pre>	<pre>1 4 2 2 0 1</pre>	-1
<pre>int rows, cols; cin >> rows >> cols; vector<vector<int>> grid(rows, vector<int>(cols)); for(int i = 0; i < rows; i++) { for(int j = 0; j < cols; j++) cin >> grid[i][j]; } cout << secondsToBeRotten(grid);</pre>	<pre>3 3 0 1 2 0 1 2 2 1 1</pre>	1



Answer: (penalty regime: 0 %)

Reset answer

```

1 int secondsToBeRotten(vector<vector<int>>& grid) {
2     int rows = grid.size();
3     int cols = grid[0].size();
4     queue<pair<int, int>> q;
5     int fresh_count = 0;
6
7     // Initialize the queue with all the rotten apples and count fresh apples
8     for (int i = 0; i < rows; ++i) {
9         for (int j = 0; j < cols; ++j) {
10            if (grid[i][j] == 2) {
11                q.push({i, j});
12            } else if (grid[i][j] == 1) {
13                fresh_count++;
14            }
15        }
16    }
17
18    if (fresh_count == 0) return 0; // No fresh apples to rot
19
20    int directions[4][2] = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
21    int time = 0;
22
23    while (!q.empty()) {
24        int q_size = q.size();
25        for (int i = 0; i < q_size; ++i) {
26            pair<int, int> p = q.front();
27            int x = p.first;
28            int y = p.second;
29            q.pop();
30            for (auto& dir : directions) {
31                int new_x = x + dir[0];
32                int new_y = y + dir[1];
33                if (new_x >= 0 && new_x < rows && new_y >= 0 && new_y < cols && grid[new_x][new_y] == 1) {
34                    grid[new_x][new_y] = 2;
35                    fresh_count--;
36                    q.push({new_x, new_y});
37                }
38            }
39        }
40        if (!q.empty()) time++;
41    }
42
43    return fresh_count == 0 ? time : -1;
44 }

```

	Test	Input	Expected	Got	
✓	<pre> int rows, cols; cin >> rows >> cols; vector<vector<int>> grid(rows, vector<int>(cols)); for(int i = 0; i < rows; i++) { for(int j = 0; j < cols; j++) cin >> grid[i][j]; } cout << secondsToBeRotten(grid); </pre>	<pre> 1 4 2 2 0 1 </pre>	-1	-1	✓
✓	<pre> int rows, cols; cin >> rows >> cols; vector<vector<int>> grid(rows, vector<int>(cols)); for(int i = 0; i < rows; i++) { for(int j = 0; j < cols; j++) cin >> grid[i][j]; } cout << secondsToBeRotten(grid); </pre>	<pre> 3 3 0 1 2 0 1 2 2 1 1 </pre>	1	1	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.



Câu hỏi 7

Đúng

Đạt điểm 1,00 trên 1,00

Given an array of integers.

Your task is to implement a function with following prototype:

```
int sumOfMaxSubarray(vector<int>& nums, int k);
```

The function returns the sum of the maximum value of every consecutive subarray of `nums` with fixed length `k`.

Note:

- The `iostream`, `vector`, `queue` and `deque` libraries have been included and `namespace std` is being used. No other libraries are allowed.
- You can write helper functions and classes.

For example:

Test	Result
<pre>vector<int> nums {1, 2, 4, 3, 6}; int k = 3; cout << sumOfMaxSubarray(nums, k);</pre>	14

Answer: (penalty regime: 0 %)

[Reset answer](#)

```

1  int sumOfMaxSubarray(vector<int>& nums, int k) {
2      if (nums.empty() || k <= 0) return 0;
3
4      deque<int> deq;
5      int sum = 0;
6
7      for (int i = 0; i < nums.size(); ++i) {
8          // Remove elements out of this window
9          while (!deq.empty() && deq.front() < i - k + 1) {
10             deq.pop_front();
11         }
12
13         // Remove elements smaller than the current element
14         while (!deq.empty() && nums[deq.back()] < nums[i]) {
15             deq.pop_back();
16         }
17
18         // Add the current element at the end of the deque
19         deq.push_back(i);
20
21         // Add the maximum element of the current window to the sum
22         if (i >= k - 1) {
23             sum += nums[deq.front()];
24         }
25     }
26
27     return sum;
28 }
```


	Test	Expected	Got	
✓	<pre>vector<int> nums {1, 2, 4, 3, 6}; int k = 3; cout << sumOfMaxSubarray(nums, k);</pre>	14	14	✓
✓	<pre>vector<int> nums {8016}; int k = 1; cout << sumOfMaxSubarray(nums, k);</pre>	8016	8016	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

